

Bro, I got you. Let's make this super clean and simple so you can just copy, paste, and get this working instantly.

Here are the exact, step-by-step instructions to set up your Pi so it automatically listens for your physical switch (on GPIO 23) every time you plug it into the wall.

Step 1: Create the Launcher File

First, we need to create the lightweight script that sits in the background waiting for you to press the button.

- Open your Kali terminal and make sure you are in your project folder (I am assuming it is `/home/kali/Desktop/TY`).
- Run this command to create the file: `nano launcher.py`
- Paste this exact code inside:

Python

```
from gpiozero import Button
import subprocess
from signal import pause

# Connected to physical switch on GPIO 23
button = Button(23, bounce_time=0.5)

server_process = None

def toggle_server():
    global server_process

    # --- TURN ON ---
    if server_process is None or server_process.poll() is not None:
        print("SWITCH PRESSED: Booting up CrowdSense AI...")
        server_process = subprocess.Popen([
            "/home/kali/Desktop/TY/.env/bin/python",
            "/home/kali/Desktop/TY/pi_serverWc4.py"
        ])

    # --- TURN OFF ---
    else:
        print("SWITCH PRESSED: Shutting down CrowdSense AI (Waiting up to 10s)...")
        server_process.terminate()

    try:
        server_process.wait(timeout=10.0)
        print("Server successfully shut down gracefully!")
    except subprocess.TimeoutExpired:
        print("10 seconds passed! Server froze. Forcing pkill...")
        server_process.kill()
```

```

subprocess.run(["sudo", "pkill", "-f", "pi_serverWc4.py"])

server_process = None
print("System resting. Waiting for switch on GPIO 23...")

print("Hardware standby mode active. Waiting for switch on GPIO 23...")
button.when_pressed = toggle_server
pause()

```

- Save and exit (Press `Ctrl+X`, then type `Y`, then hit `Enter`).

Step 2: Create the Auto-Boot Service

Next, we tell the Raspberry Pi's core system to automatically run that `launcher.py` file the second the Pi gets power.

- In your terminal, run this command to create the system file: `sudo nano /etc/systemd/system/crowdsense.service`
- Paste this exact configuration inside:

Ini, TOML

[Unit]

`Description=CrowdSense Hardware Switch Listener`

`After=network-online.target`

`Wants=network-online.target`

[Service]

`Type=simple`

`User=root`

`WorkingDirectory=/home/kali/Desktop/TY`

`ExecStart=/usr/bin/python3 /home/kali/Desktop/TY/launcher.py`

`Restart=always`

`RestartSec=5`

`StandardOutput=syslog`

`StandardError=syslog`

`SyslogIdentifier=crowdsense_launcher`

[Install]

`WantedBy=multi-user.target`

- Save and exit (Press `Ctrl+X`, then type `Y`, then hit `Enter`).

Step 3: Activate the System

Finally, we just lock the service into the Pi's memory so it runs forever.

- Run this command to reload the system: `sudo systemctl daemon-reload`
- Run this command to enable it on startup: `sudo systemctl enable crowdsense.service`
- Run this command to start it right now: `sudo systemctl start crowdsense.service`

Step 4: The Hackathon Test

You are completely done. You can close the terminal.

To test it like it's the real hackathon day:

1. Unplug the Raspberry Pi from the wall.
2. Plug it back in.
3. Wait about 45 seconds for it to boot up and connect to Wi-Fi.
4. **Press the switch on your breadboard.** Your green LED will light up, the Pi fan will kick in, and the `pi_server.py` code will blast the 1080p feed to your Flutter app. Press it again, and it safely goes to sleep! Let me know when you get it running.